6710503739_HW#8_Exercise-2

September 6, 2026

[7]:
```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

transactions = [
    ["F", "G", "I", "J"],
    ["G", "H", "J"],
    ["G", "I", "J"],
    ["G", "H", "J"],
    ["G", "H", "I", "J"],
    ["G", "I", "J"],
]

te = TransactionEncoder()
df = pd.DataFrame(te.fit_transform(transactions), columns=te.columns_,
                  index=[f"T{i}" for i in range(1, 7)])
df
```

[7]:

	F	G	H	I	J
T1	True	True	False	True	True
T2	False	True	True	False	True
T3	False	True	False	True	True
T4	False	True	True	False	True
T5	False	True	True	True	True
T6	False	True	False	True	True

0.1 1. Frequent itemsets (min support = 50%)

[8]:
```
MIN_SUP = 0.50
MIN_CONF = 0.75

freq = apriori(df, min_support=MIN_SUP, use_colnames=True)
freq["itemset"] = freq.itemsets.apply(lambda s: "{" + ", ".join(sorted(s)) +␣
 ↪"}")
freq["length"] = freq.itemsets.apply(len)
freq["count"] = (freq.support * len(transactions)).round().astype(int)
```

```
result = freq.sort_values(["length", "itemset"])[["itemset", "length", "count",␣
 ↪"support"]]
result = result.reset_index(drop=True)
print(f"Total frequent itemsets: {len(result)}\n")
result
```

```
Total frequent itemsets: 11
```

[8]:

	itemset	length	count	support
0	{G}	1	6	1.000000
1	{H}	1	3	0.500000
2	{I}	1	4	0.666667
3	{J}	1	6	1.000000
4	{G, H}	2	3	0.500000
5	{G, I}	2	4	0.666667
6	{G, J}	2	6	1.000000
7	{H, J}	2	3	0.500000
8	{I, J}	2	4	0.666667
9	{G, H, J}	3	3	0.500000
10	{G, I, J}	3	4	0.666667

0.2 2. Strong association rules

[10]:
```
rules = association_rules(freq, metric="confidence", min_threshold=MIN_CONF)
fmt = lambda s: ", ".join(sorted(s))
rules["rule"] = rules.antecedents.apply(fmt) + " -> " + rules.consequents.
 ↪apply(fmt)
table = rules[["rule", "support", "confidence", "lift"]].sort_values(
["support", "confidence"], ascending=False).reset_index(drop=True)
table.round(4)
```

```
/home/panguin/Projects/data mining/HW1/.venv/lib/python3.14/site-
packages/mlxtend/frequent_patterns/association_rules.py:184: RuntimeWarning:
invalid value encountered in divide
  cert_metric = np.where(certainty_denom == 0, 0, certainty_num /
certainty_denom)
```

[10]:

	rule	support	confidence	lift
0	G -> J	1.0000	1.0	1.0
1	J -> G	1.0000	1.0	1.0
2	I -> G	0.6667	1.0	1.0
3	I -> J	0.6667	1.0	1.0
4	G, I -> J	0.6667	1.0	1.0
5	I, J -> G	0.6667	1.0	1.0
6	I -> G, J	0.6667	1.0	1.0
7	H -> G	0.5000	1.0	1.0
8	H -> J	0.5000	1.0	1.0

```
9   G, H -> J   0.5000          1.0   1.0
10  H, J -> G   0.5000          1.0   1.0
11  H -> G, J   0.5000          1.0   1.0
```

0.2.1 3 examples of strong association rules

#	Rule	Support	Confidence	Lift
1	{H} => {G, J}	0.500	1.000 (100%)	1.000
2	{G, I} => {J}	0.667	1.000 (100%)	1.000
3	{I} => {G}	0.667	1.000 (100%)	1.000